

```
import hashlib
import time
```

```
def sha256_string(s):
    return hashlib.sha256(s.encode("utf-8")).hexdigest()
```

```
#Testing
if __name__ == "__main__":
    print(sha256_string("PhenyoMFE"))
    print(sha256_string("phenyomfe"))
    print(sha256_string("IamsuperHungry"))
    print(sha256_string("IAMSUPERhungry"))
```

```
acdc93c838402ea5bf37240da6c1654419baaba146bf791a03155366b6d3fd8e
a0c8c8e32804bedb0fff53f835bc4bd7ee48be6bc8ed22fe2c1771dd5bf10e35
26280cc4a45e9cdbee77ee959a28f625c8c0e65cfda49317a54e932e5a542487
fe1ac603fa7ef2b45afb60bee959d355fc8cd48d893a9224a1c73dbcb83ac6e9
```

```
def sha256_file(file_path):
    h = hashlib.sha256()
    with open(file_path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b''):
            h.update(chunk)
    return h.hexdigest()
```

Before changing a comma placement in the file

```
my_hash = sha256_file("/content/BLOCKCHAINASSIGNMENTDOC.rtf")
print(my_hash)
```

```
a856ecfb42511660b36f6d9c9d79b0e04a4bf17c8658ce5f4abeb0130a3ad1c0
```

After changing a comma placement in the file :)

```
my_hash = sha256_file("/content/BLOCKCHAINASSIGNMENTDOC.rtf")
print(my_hash)
```

```
f36796269540810fd6ce70934ecad93bacef36d077004727fb06f1eebb8c162c
```

```
def avalanche_demo(s):
    digest_original = hashlib.sha256(s.encode("utf-8")).hexdigest()
    mutated_s = s[:-1] + chr(ord(s[-1]) ^ 1) if s else "a"
    digest_mutated = hashlib.sha256(mutated_s.encode("utf-8")).hexdigest()
    return (digest_original, digest_mutated)
```

```
def bit_difference(hex1, hex2):
    int1 = int(hex1, 16)
    int2 = int(hex2, 16)
    xor_result = int1 ^ int2
    return bin(xor_result).count("1")
```

```
if __name__ == "__main__":
    orig, mut = avalanche_demo("PhenyoMFE")
    diff_bits = bit_difference(orig, mut)
    percent = (diff_bits / 256) * 100
    print("Original:", orig)
    print("Mutated: ", mut)
    print("Identical?", orig == mut)
    print(f"Bits different: {diff_bits}/256 ({percent:.2f}%)")
```

```
Original: acdc93c838402ea5bf37240da6c1654419baaba146bf791a03155366b6d3fd8e
Mutated: 98ea381d21fa7da5232c1a64f8d4b06977249deffa454b1af5ae58220616e946
Identical? False
Bits different: 124/256 (48.44%)
```

```
if __name__ == "__main__":
    print(sha256_string("hello"))
    print(sha256_file("/content/BLOCKCHAINASSIGNMENTDOC.rtf"))

    orig, mut = avalanche_demo("IamsuperHungry")
```

```
diff_bits = bit_difference(orig, mut)
percent = (diff_bits / 256) * 100
print("Original:", orig)
print("Mutated: ", mut)
print("Identical?", orig == mut) #True or false
print(f"Bits different: {diff_bits}/256 ({percent:.2f}%)")
```

```
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
f36796269540810fd6ce70934ecad93bacef36d077004727fb06f1eebb8c162c
Original: 26280cc4a45e9cdbee77ee959a28f625c8c0e65cfda49317a54e932e5a542487
Mutated: 91bf541dee00f7af7b29adc16cc9c57bd26739e17cdf2ae5eee0cefc43723511
Identical? False
Bits different: 139/256 (54.30%)
```

```
def proof_of_work(message, n):
    target_prefix = "0" * n
    nonce = 0
    start_time = time.perf_counter()

    while True:
        candidate = f"{message}{nonce}"
        digest = hashlib.sha256(candidate.encode("utf-8")).hexdigest()

        if digest.startswith(target_prefix):
            elapsed_seconds = time.perf_counter() - start_time
            return {
                "nonce": nonce,
                "digest": digest,
                "attempts": nonce + 1,
                "seconds": elapsed_seconds
            }

        nonce += 1
```

```
result3 = proof_of_work("PhenyoMFE", 3)
print(result3)

result4 = proof_of_work("PhenyoMFE", 4)
print(result4)
```

```
{'nonce': 606, 'digest': '0003891c620d0776ee4b36f72903814768d7f03db78db0ed3659681ebddf9cc6', 'attempts': 607, 'se
{'nonce': 18392, 'digest': '00002a3b8eb311a2c5fcfb7a72a210f6bbb9d91f835e3717da0e6fba49f20105', 'attempts': 18393,
```